

Smart Intelligence: A Deterministic In-Browser Product Discovery System with Natural-Language Search and Explainable Ranking

A Technical Research Paper on Frontend Architecture, Information Retrieval, and E-Commerce UX Engineering

Himanshu

Independent Software Engineering Project

Repository: github.com/Himanshu90909/Smart-Intelligence | September 2026

Abstract

This paper presents *Smart Intelligence*, a React 19 and TypeScript product-discovery application that implements a fully deterministic, client-side search engine for e-commerce product catalogs. Unlike generative-AI approaches, the system combines classical information-retrieval techniques—TF-IDF vectorization and cosine similarity—with a natural-language filter parser that interprets queries such as “white running shoes under \$3000.” The architecture emphasizes explainable ranking, URL-synchronized faceted search, optimistic cart updates, product comparison with attribute-derived verdicts, and an optional server-side proxy for live Amazon data. We describe the layered frontend design, accessibility and performance decisions, testing strategy (Vitest + Playwright), and deliberate technology trade-offs. Evaluation focuses on engineering transparency rather than unverified accuracy claims: an Engineering page reports real-time vector-build, search, and Web-Vitals measurements. The work demonstrates that classical IR methods, carefully engineered UX, and strict secret boundaries can deliver a production-grade shopping experience without relying on opaque large-language-model inference.

Keywords: product discovery, TF-IDF, cosine similarity, natural-language search, React, TypeScript, faceted filtering, e-commerce UX, explainable ranking, client-side IR

1. Introduction

Modern e-commerce platforms increasingly rely on machine-learning recommenders and large language models for product search and personalization. While powerful, these systems often lack transparency: ranking decisions are difficult to explain, training data may introduce bias, and inference costs scale with traffic. In contrast, classical information-retrieval (IR) techniques such as term-frequency-inverse-document-frequency (TF-IDF) and cosine similarity remain fully deterministic, auditable, and computationally inexpensive when applied to modest catalog sizes.

Smart Intelligence is a deliberate exploration of this classical path. The application delivers an Amazon-style product discovery experience—natural-language queries, faceted filters, comparison, cart, and wishlist—while keeping the entire ranking pipeline inside the browser. The design goals are:

- **Determinism** — identical queries always produce identical ranked lists;
- **Explainability** — recommendation reasons are derived from concrete product attributes;
- **Privacy and cost** — core search requires no network round-trip or external model;
- **Engineering honesty** — the system never claims generative-AI capabilities it does not possess;
- **Production readiness** — accessibility, responsive layout, testing, and safe secret handling are first-class concerns.

The remainder of this paper is organized as follows. Section 2 reviews related work in client-side search and e-commerce UX. Section 3 details the system architecture and information-retrieval pipeline. Section 4 covers the natural-language parser and faceted filtering. Section 5 describes the frontend engineering decisions. Section 6 outlines the testing and deployment strategy. Section 7 discusses limitations and future work. Section 8 concludes.

2. Related Work

Client-side full-text search has a long history. Libraries such as Lunr.js and FlexSearch bring inverted-index techniques into the browser, typically for documentation sites or small catalogs. Elasticsearch and Algolia dominate server-side product search; both support faceting, synonym expansion, and relevance tuning, but require infrastructure and introduce latency. Recent work on “zero-shot” text classification with large language models (Wang et al., 2024) shows that LLMs can act as flexible classifiers, yet they remain non-deterministic and costly for interactive ranking.

On the UX side, Amazon’s product-detail and comparison pages set the de-facto standard for attribute-rich shopping interfaces. Academic studies of faceted search (Hearst, 2006) emphasize the value of progressive refinement and the risk of empty result sets—problems addressed here by a graceful facet-relaxation path. Accessibility guidelines (WCAG 2.2) and Core Web Vitals have become non-negotiable for production storefronts; Smart Intelligence incorporates both from the outset.

3. System Architecture

3.1 Layered Frontend Design

The application is structured so that UI, domain logic, persistence, and external data boundaries remain independently replaceable. Table 1 summarizes the layers.

Layer	Responsibility	Examples
Routes / pages	Compose user journeys and route-level loading	src/pages, src/App.tsx
Reusable UI	Cards, drawers, modals, empty/error/loading states	src/components
Domain logic	Search, parsing, filters, recommendations, verdicts	src/lib
State	Shared client state persisted to localStorage	src/store
Data boundary	Typed live-data client and server-side proxy	src/lib/amazon.ts, functions/
Static data	Bundled product catalog and derived facets	src/data

Table 1. Layered architecture of Smart Intelligence.

3.2 Information-Retrieval Pipeline

At application start (or after catalog reload) a single memoized vector index is built. Each product is treated as a document whose text fields—title, brand, category, color, material, and description—are tokenized, lower-cased, and converted into a TF-IDF vector. Query vectors are computed identically. Ranking is performed by cosine similarity:

$$\text{sim}(q, d) = (q \cdot d) / (\|q\| \|d\|)$$

Because the catalog contains only a few hundred records, full pairwise comparison is inexpensive and avoids the need for approximate nearest-neighbor indexes. When a natural-language query also expresses hard constraints (price ceiling, color, brand, etc.), those constraints are applied first as Boolean filters; cosine ranking then orders the surviving set. If the exact facet combination yields zero results, a controlled relaxation path successively drops the least-specific facets until matches appear, preserving user intent as far as possible.

3.3 API and Secret Boundary

Live Amazon search is optional and is never required for core functionality. The browser communicates exclusively with a same-origin route `/api/amazonLive`. A server-side function holds the `RAPIDAPI_KEY`, validates request modes, enforces timeouts, normalizes payloads, and never surfaces upstream stack traces. This

design eliminates the classic anti-pattern of embedding third-party credentials in client bundles.

4. Natural-Language Parsing and Faceted Search

Users may type free-form expressions such as “white running shoes under \$3000” or “Nike sneakers rating above 4.” A lightweight parser extracts structured filter predicates—color, brand, category, price range, minimum rating, discount, material, availability—while residual tokens form the free-text query vector. Parsed state is written into the URL query string, enabling shareable links and browser history navigation. All facet controls (checkboxes, range sliders, sort selects) remain synchronized with the same URL state, so the interface is always a pure function of the current location.

Recommendation rails displayed on product-detail pages derive their “why” explanations from overlapping attribute sets (same brand, similar price band, matching color family) rather than from opaque model scores. This design choice keeps the system honest: every ranking reason can be inspected by a developer or end user.

5. Frontend Engineering Decisions

5.1 Technology Choices

React 19 was selected for its composable component model, mature testing ecosystem, and predictable state boundaries. TypeScript supplies static typing for domain models and search algorithms. React Router enables client-side navigation with route-level code splitting via dynamic imports, keeping the initial bundle small. State that must survive route changes—cart, wishlist, comparison list, recently viewed, search history—is managed by a minimal Context layer backed by localStorage; heavier libraries such as Redux were deliberately omitted to reduce dependency surface.

jQuery is not used inside React-controlled subtrees: concurrent DOM mutation would create two sources of truth. Third-party packages are limited to routing, unit testing (Vitest + Testing Library), and end-to-end testing (Playwright). No UI component framework, charting library, or artificial keyword dependency was introduced without a concrete product need.

5.2 Accessibility and Responsiveness

Semantic HTML, labelled form controls, keyboard-operable overlays, focus-visible outlines, and `aria-live` regions are applied consistently. Images carry meaningful alt text and sit inside fixed aspect-ratio containers to minimize cumulative layout shift. On narrow viewports a filter drawer replaces the desktop sidebar, preserving full facet functionality without horizontal scroll.

5.3 Performance

Route-level lazy loading, debounced search suggestions, lazy image loading, and a single memoized vector index keep computational cost proportional to the current interaction. Because the catalog is small, pagination is preferred over virtualization for simplicity and maintainability. An Engineering page surfaces real measurements—vector-build time, search latency, Web Vitals, navigation timing, and resource sizes—collected in the current browser session, rather than publishing unverified claims.

6. Testing and Deployment

Unit tests (Vitest) cover vector ranking, filter logic, pagination, formatting helpers, recommendation scoring, and comparison-verdict generation. Playwright end-to-end suites exercise catalog search and filtering, product-to-cart flows, comparison interactions, and natural-language interpretation. The continuous-integration workflow runs the full unit suite on every push; E2E tests are executed against a production build preview.

Deployment targets Vercel. The standard Vite build command produces a static asset directory; serverless functions host the Amazon proxy. Environment variables holding secrets are never prefixed with `VITE_`, guaranteeing they remain server-side. SPA rewrites exclude the `/api` namespace so that the proxy route is never swallowed by the client-side fallback.

Table 2 maps the implemented features onto a typical front-end engineer skill rubric (illustrated with an Amazon internship-style checklist).

Skill Area	Evidence in Project
JavaScript / TypeScript	Typed domain models, deterministic search algorithms, React application logic
HTML / CSS	Semantic controls, responsive design system, accessible overlays and forms
React	Component architecture, Context, hooks, route-level lazy loading
Reusable UX components	Product cards, drawers, modals, filter controls, pagination, feedback states
REST API integration	Centralized Amazon client, timeout/error handling, server-side proxy
UX patterns	Loading, error, empty, retry, reset, search, filter, sort, pagination, compare
Testing	Vitest unit tests, React Testing Library, Playwright critical user flows
CS fundamentals	TF-IDF indexing, cosine similarity, filtering, pagination, deterministic ranking
Operational excellence	Build validation, safe errors, secret boundary, SPA/API routing separation

Table 2. Mapping of project artifacts to front-end engineering competencies.

7. Limitations and Future Work

The current catalog is static and fully client-side; cross-device account synchronization, server-backed carts, authentication, and persistent analysis history are not implemented. Live Amazon results depend on a third-party API, its rate limits, and a correctly configured server-side key. The search engine does not claim generative capabilities—its value lies in transparent, reproducible ranking.

Future directions include: (1) incremental index updates when new products are added, (2) synonym and stemming expansions to improve recall, (3) A/B instrumentation of ranking variants, (4) progressive web-app offline support, and (5) optional hybrid ranking that blends classical scores with lightweight on-device embeddings while preserving explainability.

8. Conclusion

Smart Intelligence demonstrates that a carefully engineered classical information-retrieval pipeline, combined with modern React practices and strict operational discipline, can produce a rich, accessible, and explainable product-discovery experience entirely inside the browser. By refusing to over-claim generative-AI abilities and by exposing real performance measurements, the project prioritizes engineering honesty. The resulting system serves both as a usable shopping prototype and as a concrete portfolio artifact illustrating front-end architecture, IR fundamentals, testing rigor, and secure API design.

References

- [1] Hearst, M. A. (2006). Clustering versus faceted categories for information exploration. *Communications of the ACM*, 49(4), 59–61.
- [2] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [3] Wang, Z., Pang, Y., & Lin, Y. (2024). Smart Expert System: Large Language Models as Text Classifiers. arXiv:2405.10523.
- [4] React Team. (2024). React 19 Documentation. <https://react.dev>
- [5] Vite Team. (2024). Vite Documentation. <https://vitejs.dev>
- [6] Microsoft. (2024). Playwright Documentation. <https://playwright.dev>
- [7] W3C. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>
- [8] Google. (2024). Web Vitals. <https://web.dev/vitals/>
- [9] Himanshu. (2026). Smart Intelligence source repository. <https://github.com/Himanshu90909/Smart-Intelligence>